Hierarchical LeWorldModel

End-to-End Two-Level Latent World Modeling
with Latent Actions, CEM Planning, and SIGreg Regularization

April 17, 2026

Abstract

We propose **Hierarchical LeWorldModel (H-LeWM)**, a fully end-to-end, multi-level latent world model architecture that extends LeWorldModel (LeWM) long-horizon goal conditioned control. Both the low-level and high-level world models are instantiated as LeWM modules, trained jointly from scratch without any pretrained backbone and share a single visual encoder. The high-level module operates over temporally abstracted transitions via a learned *latent action encoder*, which compresses sequences of primitive actions into compact macro-actions, eliminating the need for any inverse dynamics (ID) system. At inference time, a hierarchical cross-entropy method (CEM) planner couples the two levels through shared latent subgoals: the high-level planner proposes intermediate latent states, which the low-level planner tracks via primitive actions. Anti-collapse is enforced jointly at the 2 levels by the Sketched Isotropic Gaussian Regularizer (SIGreg), and the entire system is optimized end-to-end with a simple three term object. H-LeWM addresses the long-horizon brittleness of flat world-model planners while preserving the training simplicity and stability that characterize the original LeWM.

1 Introduction

World models that learn to predict future latent states from raw pixels are a powerful substrate for goal-conditioned control (**?**). The Joint-Embedding Predictive Architecture (JEPA) family learns compact representations by predicting future embeddings conditioned on actions, without requiring pixel reconstruction. LeWorldModel (LeWM) (**?**) demonstrated that a two-term objective—a predictive loss and a SIGreg anti-collapse regularizer—is sufficient to train a stable JEPA-style world model entirely end-to-end from raw pixels on a single GPU, outperforming methods that rely on pretrained encoders or heuristic multi-term objectives.

Despite these strengths, flat world-model planners degrade rapidly as task horizons grow. Compounding prediction errors and an exponentially expanding search space make long-horizon CEM optimization unreliable (**?**). Hierarchical Planning with Latent World Models (HWM) (**?**) showed that coupling a high-level and a low-level world model through shared latent subgoals—while using a learned latent action encoder at the high level—substantially improves long-horizon performance and reduces planning compute. However, HWM reuses pretrained low-level backbones (VJEPA2-AC, DINO-WM, PLDM) and does not train the hierarchy end-to-end.

We close this gap with **Hierarchical LeWorldModel (H-LeWM)**:

- **Both levels are LeWM modules**, sharing the same ViT encoder and trained jointly from scratch.
- **Latent macro-actions** replace raw action sequences at the high level, compressing temporal structure without inverse dynamics models.

- **CEM couples the hierarchy at inference time** through latent subgoal matching, requiring no task-specific rewards or learned policies.
- The full system is trained **end-to-end**, with no pretraining stage, frozen weights, or heuristic stabilization.

2 Background

LeWorldModel. LeWM (**?**) consists of a Vision Transformer (ViT) encoder f_θ and a transformer predictor g_ϕ. Given a sequence of raw pixel observations $o_{1:T}$ and actions $a_{1:T}$, the encoder maps each frame to a latent embedding $z_t = f_\theta(o_t)$, and the predictor estimates the next embedding as $\hat{z}_{t+1} = g_\phi(z_t, a_t)$. The training objective is

$$\mathcal{L}_{\text{LeWM}} = \mathcal{L}_{\text{pred}} + \lambda\, \mathcal{L}_{\text{SIGreg}}, \tag{1}$$

where $\mathcal{L}_{\text{pred}} = \|z_{t+1} - \hat{z}_{t+1}\|_2^2$ is the next-embedding prediction loss and

$$\mathcal{L}_{\text{SIGreg}}(Z) = \frac{1}{M} \sum_{m=1}^{M} T\Big(h_m Z^\top u_m\Big), \tag{2}$$

with $\{u_m\}_{m=1}^M \subset \mathcal{S}^{d-1}$ random unit-norm projections and $T(\cdot)$ the univariate Epps–Pulley test statistic measuring deviation from a standard normal. By the Cramér–Wold theorem, matching all one-dimensional marginals implies matching the full joint distribution to an isotropic Gaussian, preventing representation collapse.

Hierarchical Planning with Latent World Models. HWM (**?**) introduced a two-level inference-time hierarchy over pretrained world models. A high-level model P_2 is trained to predict waypoint-level transitions conditioned on *latent macro-actions* l_t, which are produced by a transformer action encoder $\mathcal{A}$ that compresses sequences of primitive actions between waypoints. At inference, the high-level CEM produces intermediate latent subgoals; the low-level CEM then plans primitive actions to reach each subgoal. The two levels share the same latent space and encoder, enabling direct coupling via latent-state matching.

3 Hierarchical LeWorldModel (H-LeWM)

3.1 Architecture Overview

H-LeWM consists of the following jointly-trained components:

1. A **shared ViT encoder** $f_\theta : \mathcal{O} \to \mathbb{R}^d$, mapping raw pixel frames to latent embeddings.
2. A **low-level predictor** P_1^ϕ: a transformer that predicts z_{t+1} from (z_t, a_t), operating at the primitive action timescale.
3. A **latent action encoder** $\mathcal{A}_\psi$: a transformer that encodes a sub-sequence of primitive actions $a_{t_k:t_{k+1}}$ between consecutive waypoints into a compact macro-action $l_k \in \mathbb{R}^{d_l}$.
4. A **high-level predictor** P_2^ξ: a transformer conditioned on macro-actions l_k that predicts the next waypoint embedding $z_{t_{k+1}}$ from z_{t_k}, operating at the abstract timescale.

Crucially, **no inverse dynamics (ID) model is needed**: macro-actions l_k are produced by the forward action encoder $\mathcal{A}_\psi$ at training time, and at inference time the high-level CEM directly optimizes over the latent action space $\mathbb{R}^{d_l}$ without requiring action inversion.

Algorithm 1 H-LeWM Training

Require: Offline dataset $\mathcal{D}$, waypoints N, projections M, weights λ, α, β

```
while not converged do
    Sample mini-batch {(o_{1:T}, a_{1:T})} ~ D
    Encode all frames: z_t ← f_θ(o_t)                          // shared encoder
    Compute L_SIGreg on {z_t}                                  // single anti-collapse term
    Low-level branch:
        ẑ_{t+1} ← P_1^φ(z_t, a_t) for all t
        Compute L_pred^(1) via eq. (3)
    High-level branch:
        Sample waypoint indices t_1 < ··· < t_N
        l_k ← A_ψ(a_{t_k:t_{k+1}}) for all k                   // no ID model needed
        ẑ_{t_{k+1}} ← P_2^ξ(z_{t_k}, l_k) for all k
        Compute L_pred^(2) via eq. (4)
    Compute L_H-LeWM via eq. (5)
    Backpropagate through {θ, φ, ψ, ξ} jointly
end while
```

3.2 End-to-End Training Objective

Let $\mathcal{D} = \{(o_{1:T}, a_{1:T})\}$ be an offline, reward-free dataset of trajectories. We sample N waypoint indices $1 \leq t_1 < t_2 < \cdots < t_N \leq T$ per trajectory. The training objective combines four terms:

Low-Level Prediction Loss.

$$\mathcal{L}_{\text{pred}}^{(1)} = \frac{1}{T-1} \sum_{t=1}^{T-1} \|z_{t+1} - \hat{z}_{t+1}\|_2^2, \quad \hat{z}_{t+1} = P_1^{\phi}(z_t, a_t), \tag{3}$$

where $z_t = f_\theta(o_t)$ are encoder outputs (teacher-forced).

High-Level Prediction Loss.

$$\mathcal{L}_{\text{pred}}^{(2)} = \frac{1}{N-1} \sum_{k=1}^{N-1} \left\|z_{t_{k+1}} - \hat{z}_{t_{k+1}}\right\|_2^2, \quad \hat{z}_{t_{k+1}} = P_2^{\xi}(z_{t_k},\ l_k)\,, \tag{4}$$

where $l_k = \mathcal{A}_\psi(a_{t_k:t_{k+1}})$ is the macro-action produced by the action encoder (CLS token projection).

Full Objective.

$$\boxed{\mathcal{L}_{\text{H-LeWM}} = \alpha\ \mathcal{L}_{\text{pred}}^{(1)} + \beta\, \mathcal{L}_{\text{pred}}^{(2)} + \lambda\, \mathcal{L}_{\text{SIGreg}}} \tag{5}$$

All parameters $\{\theta, \phi, \psi, \xi\}$ are optimized jointly; no stop-gradients, EMA targets, or frozen components are used. Anti-collapse regularization is applied to the encoder shared by both levels. By default α and β we would start by setting them both to 1.

3.3 Latent Action Encoder and Removal of Inverse Dynamics

The latent action encoder $\mathcal{A}_\psi$ is a bidirectional transformer with 2 layer, dim=192, heads=4, mlp_dim = 768, dropout = 0.1. Given the sub-sequence $a_{t_k}, a_{t_k+1}, \ldots, a_{t_{k+1}-1}$ of primitive actions between two waypoints, a learnable `[CLS]` token is prepended, and the transformer

processes the sequence bidirectionaly. The [CLS] output is projected through a one-layer MLP to obtain $l_k \in \mathbb{R}^{d_l}$:

$$l_k = \mathrm{MLP}\big(\mathcal{A}_\psi\big([\texttt{CLS};\, a_{t_k:t_{k+1}}]\big)_{\texttt{CLS}}\big)\,. \tag{6}$$

At *inference time*, no primitive actions are available at the high level; instead, the CEM optimizer directly samples candidate macro-actions $l \sim \mathcal{N}(\mu, \sigma^2 I)$ in $\mathbb{R}^{d_l}$ and evaluates them through the high-level predictor P_2^ξ. This renders the high-level planning problem:

$$l^*_{1:H} =_{l_{1:H} \in \mathbb{R}^{d_l \times H}} \left\| P_2^\xi(l_{1:H}, z_1) - z_g \right\|_2^2, \tag{7}$$

where $z_g = f_\theta(o_g)$ is the goal embedding. The first predicted latent state $\hat{z}_{t_1} = P_2^\xi(z_1, l_1^*)$ serves as the **subgoal** for the low-level planner.

The absence of an inverse dynamics model is a key design choice: ID systems introduce additional training complexity and require separate optimization targets. By learning $\mathcal{A}_\psi$ as a *forward* encoder supervised purely by the high-level prediction loss, we avoid this entirely.

3.4 Hierarchical CEM Planning

At inference, H-LeWM performs top-down hierarchical Model Predictive Control (MPC), mirroring the HWM strategy (**?**) but with both world models being LeWM modules sharing the encoder f_θ.

High-Level CEM. Given current observation o_1 and goal o_g:

1. Encode: $z_1 = f_\theta(o_1)$, $z_g = f_\theta(o_g)$.
2. Initialize macro-action distribution $\mu = \mathbf{0}$, $\sigma = \mathbf{1}$.
3. Sample N_{high} candidate sequences $\{l_{1:H}\}$; roll out P_2^ξ autoregressively.
4. Evaluate goal cost $C_{\text{high}} = \|\hat{z}_H - z_g\|_2^2$; update (μ, σ) from elite set.
5. Repeat for K_{high} iterations; extract subgoal $z_{\text{sub}} = \hat{z}_1$ from best plan.

Low-Level CEM. Given current z_1 and subgoal z_{sub}:

1. Initialize primitive action distribution.
2. Sample N_{low} candidate sequences $\{a_{1:h}\}$; roll out P_1^ϕ autoregressively.
3. Evaluate subgoal cost $C_{\text{low}} = \|\hat{z}_h - z_{\text{sub}}\|_2^2$; update distribution from elite set.
4. Execute first K_{exec} actions; re-plan in MPC fashion.

When the low-level planner reaches z_{sub}, the high-level planner is queried again to produce the next subgoal, continuing until the terminal goal z_g is reached.

4 Theoretical Motivation

Latent actions and CEM compatibility. By compressing primitive action sub-sequences into a fixed-dimensional macro-action $l_k \in \mathbb{R}^{d_l}$, the high-level CEM operates in a significantly lower-dimensional space than raw action sequences, reducing optimization complexity from $\mathcal{O}(h \cdot d_a)$ to $\mathcal{O}(d_l)$ per high-level step. The action encoder absorbs the temporal complexity of the sub-sequence, making the search space tractable even for long horizons.

End-to-end gradient flow. Unlike two-stage methods, joint optimization allows the encoder f_θ to be shaped simultaneously by low-level predictability (via $\mathcal{L}_{\text{pred}}^{(1)}$) and high-level predictability (via $\mathcal{L}_{\text{pred}}^{(2)}$). This acts as an implicit multi-scale smoothness inductive bias on the latent space: the encoder is incentivized to produce embeddings that are predictable at both short and long temporal resolutions, which is a desirable property for hierarchical planning.